Hochschule Bochum
TECHNIK WIRTSCHAFT GESUNDHEIT

Objektorientierte Programmiertechniken

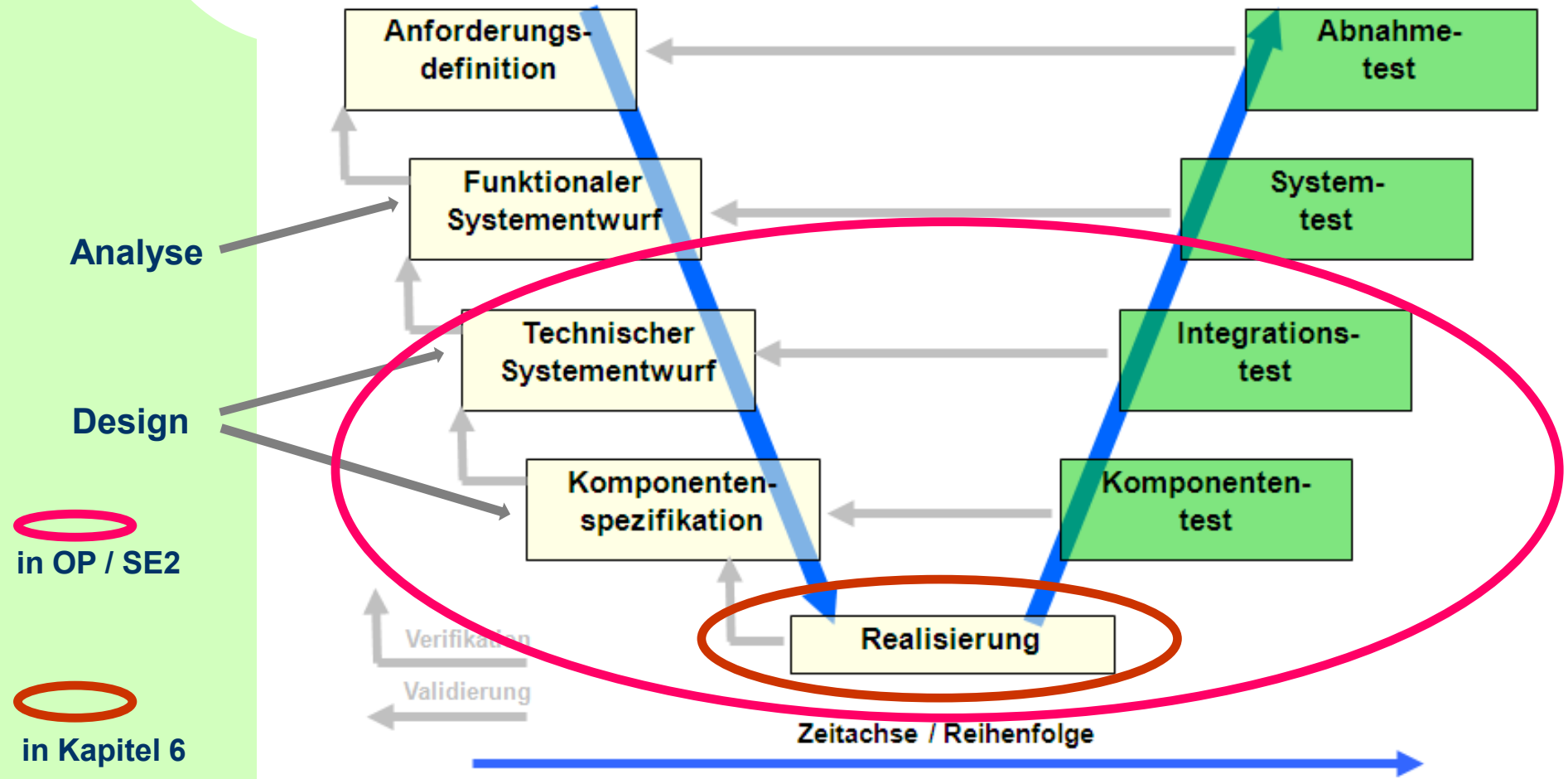
Prof. Dr. Ursula Oesing

sonstiges

- Reflection
- Reguläre Ausdrücke
- Consumer
- Streams
- gegebenenfalls ein weiteres Design Pattern

Vorspann

- Einordnung der Veranstaltung



Reflection

Was ist Reflection?

- Reflection (Reflexion) ist die Fähigkeit eines Programms, die eigene Struktur zur Laufzeit zu analysieren oder auch zu verändern.
- Mit Hilfe von Reflection können zur Laufzeit anhand eines Objektes Informationen über seine Klasse (Attribute, Konstruktoren, Methoden, Modifier, Annotationen, …) ausgelesen werden.
- Mit Hilfe von Reflection können generisch und dynamisch Konstruktoren und Methoden eines Objekts aufgerufen werden.

Reflection

Reguläre Ausdrücke

Consumer

Streams

Reflection

Erstes Beispiel zu Reflection

- siehe Lösung zu Übung 7

Der Klassen-name der Klasse des Objekts obs wird zur Laufzeit ermittelt.

```
public class SportartikelView implements Observer {
    ...
    public void update(Observable obs) {
        if(obs.getClass().getSimpleName()
                .equals("TrikotsModel")) {
                zeigeTrikotsAn();
        }
        if(obs.getClass().getSimpleName()
                .equals("BaelleModel")) {
                zeigeBaelleAn();
        }
    }
    ...
}
```


Reflection

Reguläre Ausdrücke

Consumer

Streams

Reflection

Reflection in Java: Klassen und Attribute

- Die Methode getClass() der Klasse Object gibt ein Class- Objekt zurück. Dieses enthält dann Informationen über die Klasse des Objekts, mit welchem getClass() aufgerufen wurde.
- Die Methode getName() der Klasse Class gibt ein String-Objekt zurück, welcher den Namen der Klasse inkl. packages enthält.
- Die Methode getSimpleName() der Klasse Class gibt ein String-Objekt zurück, welcher den Namen der Klasse ohne packages enthält.
- Die Methode getDeclaredFields() der Klasse Class gibt einen Array von Field- Objekten zurück, die Attribute der Klasse (ohne die geerbten Attribute).
- Die Methode getName() der Klasse Field gibt ein String-Objekt zurück, welcher den Namen des Attributs enthält.

Reflection

Reguläre Ausdrücke

Consumer

Streams

Reflection

Reflection in Java: Methoden

- Die Methode
  getMethod(String name, Class<?>... parameterTypes)
  der Klasse Class gibt ein Method- Objekt zurück, die Methode der Klasse mit dem vorgegebenen Namen name und der vorgegebenen Parameterliste parameterTypes.
- Die Methode getDeclaredMethods() der Klasse Class gibt einen Array von Method- Objekten zurück, die Methoden der Klasse (ohne die geerbten Methoden).
- Die Methode invoke(Object obj, Object... args) der Klasse Method führt die Methode mit aufrufendem Objekt obj und den vorgegebenen Parametern args aus.

Reflection

Reguläre Ausdrücke

Consumer

Streams

Reflection

Reflection in Java: Annotationen von Methoden

- Gegeben sei eine Annotation Ausgabe, mit welcher Methoden gekennzeichnet werden, nach denen zur Laufzeit gesucht werden soll.

```
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
public @interface Ausgabe {
}
```

- Die Methode isAnnotationPresent(Class<? extends Annotation> annotationClass) gibt true aus, falls eine Methode mit der vorgegebenen Annotation gekennzeichnet ist, false sonst.

```
Method methode = null;
// Initialisierung der Variablen methode
...
boolean vorhanden = methode.isAnnotationPresent(Ausgabe.class);
```

Reflection
Reguläre Ausdrücke
Consumer
Streams

Reflection

Aufgabe zu Reflection

- Gegeben seien die Klassen Werkzeug und Auto, siehe die folgenden Folien. Gegeben sei die Klasse Anwendung.

```
import java.lang.reflect.*;

public class Anwendung {

    public void gibObjektInKonsoleAus(Object obj)  {
        ……
    }

    public static void main(String[ ] args) {
        Anwendung anw = new Anwendung();
        Object obj = new Werkzeug(11, 12, "Werkzeugnorm");
        anw.gibObjektInKonsoleAus(obj);
        obj = new Auto("BO YC 123", 125);
        anw.gibObjektInKonsoleAus(obj);
    }
}
```

Reflection

Reguläre Ausdrücke

Consumer

Streams

Reflection

Fortsetzung: Aufgabe zu Reflection, die Klasse Werkzeug

Reflection

Reguläre Ausdrücke

Consumer

Streams


```
public class Werkzeug {
    private int identnummer;
    private int basispreis;
    private String werkzeugart;

    // Konstruktor, welcher die Attribute mit Werten belegt
    // get- und set-Methoden
    …

     @Ausgabe
     public String gibAttributeAus(){
          return this.getIdentnummer() + "  "
               + this.getBasispreis() + "  "
               + this.getWerkzeugart();
    }
}
```

Reflection

- **Fortsetzung: Aufgabe zu Reflection, die Klasse Auto**

Reflection

Reguläre Ausdrücke

Consumer

Streams


```
public class Auto {
    private String kennzeichen;
    private int ps;

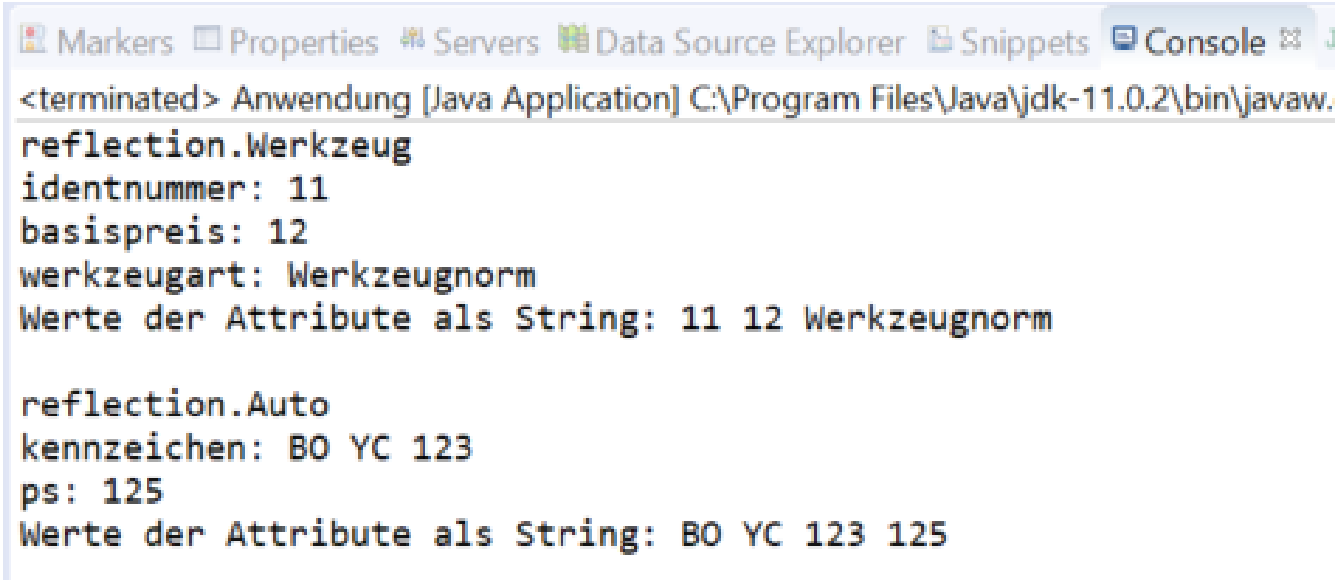
    // Konstruktor, welcher die Attribute mit Werten belegt
    // get- und set-Methoden
    …

     @Ausgabe
     public String gibAttributeAn(){
         return this.getKennzeichen() + "  "  + this.getPs();
     }
}
```

Reflection

Fortsetzung der Aufgabe zu Reflection

- Vervollständigen Sie gibObjektInKonsoleAus, so dass die folgende Konsolenausgabe erzeugt wird. Benutzen Sie Reflection. Gehen Sie davon aus, dass die vogegebenen Klassen im package reflection liegen.



Reflection

Reguläre Ausdrücke

Consumer

Streams

Reflection

Reflection

Reguläre Ausdrücke

Consumer

Streams

Wieso Reflection?

- Reflection für die generische Programmierung
  (z. B. von Datenbank-Frameworks)

  vgl.: http://javatricks.de/tricks/was-versteht-man-unter-reflection
- Reflection für Debugger, …
- Reflection für Codeanalyse – Tools
  (z. B. PMD: Feststellung, ob eine private Methode in derselben Klasse aufgerufen wird oder nicht)
- Verwendung von Annotationen in Frameworks
  (z .B. JUnit: Aufruf aller Methoden mit der Annotation @Test)

Reguläre Ausdrücke

Reflection

Reguläre Ausdrücke

Consumer

Streams

Was sind Reguläre Ausdrücke?

vgl.: https://de.wikipedia.org/wiki/Regulärer_Ausdruck

- *Ein regulärer Ausdruck (englisch regular expression, Abkürzung RegExp oder Regex) ist in der theoretischen Informatik eine Zeichenkette, die der Beschreibung von Mengen von Zeichenketten mit Hilfe bestimmter syntaktischer Regeln dient. Reguläre Ausdrücke finden vor allem in der Softwareentwicklung Verwendung.*

  …

  Reguläre Ausdrücke beschreiben eine Familie von formalen Sprachen und gehören damit zur theoretischen Informatik.

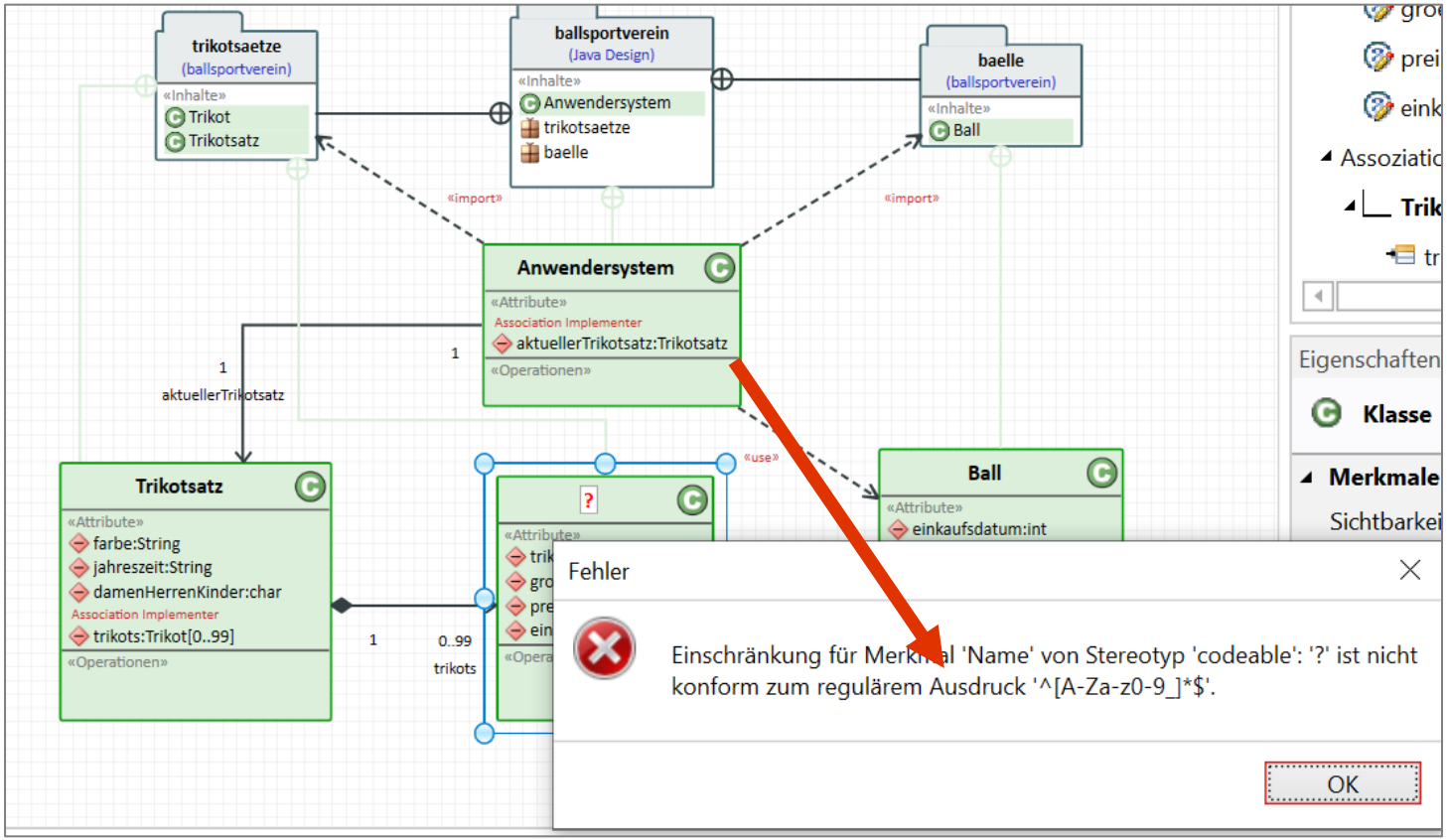
Es folgen daher nur erste Anwendungsbeispiele.

Reguläre Ausdrücke

- Reflection
- **Reguläre Ausdrücke**
- Consumer
- Streams

Wann sind Ihnen bereits Reguläre Ausdrücke begegnet?

- bei Innovator: Klassendiagramm



Reguläre Ausdrücke

Reflection

Reguläre Ausdrücke

Consumer

Streams

Beispiele von Zeichen in Regulären Ausdrücken

vgl.: http://openbook.rheinwerk-verlag.de

- . : Jedes beliebige Zeichen ist erlaubt.
- \d : Die Ziffern 0 bis 9 sind erlaubt.
- x* : Es werden beliebig viele Zeichen x erlaubt.
- [aeiou] : Die Zeichen a, e, i ,o und u sind erlaubt.
- [^aeiou] : Zeichen ungleich a, e, i ,o oder u sind erlaubt.

und viele mehr.

Beispiel aus dem Klassendiagramm von Innovator

- ^[A-Za-z0-9_]*$:
  Beliebig viele Zeichen aus der folgenden Menge:
  Großbuchstaben (außer Umlaute), Kleinbuchstaben (außer Umlaute und ß), Ziffern, Unterstrich
  sind erlaubt. ^ und $ bedeuten hier Zeilenanfang und Zeilenende.

Reguläre Ausdrücke

Reflection

Reguläre Ausdrücke

Consumer

Streams

Szenarien zu Regulären Ausdrücken

vgl.: http://openbook.rheinwerk-verlag.de

Übereinstimmungen

- Passt eine Zeichenfolge komplett auf ein Muster?
- Die Methode matches(String regex) der Klasse String gibt true zurück, falls der reguläre Ausdruck regex mit dem aktuellen String – Objekt übereinstimmt, false sonst.

Die statische Methode matches(String regex, CharSequence input) der Klasse Pattern gibt true zurück, falls der reguläre Ausdruck regex mit der vorgegebenen Zeichenkette input übereinstimmt, false sonst.

```
public final class String … {
    …
    public boolean matches(String regex) {
        return Pattern.matches(regex, this);
    }
}
```

Reguläre Ausdrücke

Reflection

Reguläre Ausdrücke

Consumer

Streams

Aufgabe, Teil 1 zu Regulären Ausdrücken

- Gegeben sei die folgende Klasse Anwendung. Ergänzen Sie diese, so dass Sie die folgende Ausgabe in der Konsole erhalten.

Wie erhalten Sie Übereinstimmung?

```
Die Zeichenkette stimmt mit dem vorgegebenen Muster
Liebe Studierende, wie geht es Ihnen?
nicht überein.
```

```
public class Anwendung {
    private String zeichenkette;
    // Konstruktor, welcher zeichenkette belegt

    private void pruefeStimmtZeichenketteUeberein(
        String regex){
        // hier ergaenzen
        …
    }

    public static void main(String[ ] args){
        Anwendung anw = new Anwendung(
            "Liebe Studierende, wie geht es Ihnen?")
        anw.pruefeStimmtZeichenketteUeberein(
            /* hier ergaenzen */ …)
    }
}
```

Reguläre Ausdrücke

- **Szenarien zu Regulären Ausdrücken**
- **Ersetzen von Teilfolgen**
  - Teilfolgen, die auf ein Muster passen, werden durch eine andere Teilfolge ersetzt.
  - Die Methode replaceAll(String regex, String replacement) der Klasse String gibt ein String- Objekt zurück, in welchem alle Teile, die zu dem regulären Ausdruck regex passen, durch den String replacement ersetzt werden.

```
public final class String … {
    …
    public String replaceAll(String regex, String replacement) {
        return Pattern.compile(regex)
            .matcher(this).replaceAll(replacement);
    }
}
```

Reflection

Reguläre Ausdrücke

Consumer

Streams

Reguläre Ausdrücke

Reflection

Reguläre Ausdrücke

Consumer

Streams

Aufgabe, Teil 2 zu Regulären Ausdrücken

- Gegeben sei die folgende Klasse Anwendung. Ergänzen Sie diese, so dass Sie die folgende Ausgabe in der Konsole erhalten.

Wie erreichen Sie auch eine Ersetzung von morgen?
Listing vormachen

```
Das Ergebnis nach Ersetzen von der Teilfolge
Morgen mit Tag ist
Guten Tag liebe Studierende!
```

```
public class Anwendung {
    private String zeichenkette;
    // Konstruktor, welcher zeichenkette belegt

    private void ersetzeTeilfolge(
        String regex, String replacement){
        // hier ergaenzen
        …
    }

    public static void main(String[ ] args){
        Anwendung anw = new Anwendung(
            "Guten Morgen liebe Studierende!")
        anw.ersetzeTeilfolge("Morgen", "Tag");
    }
}
```

Reguläre Ausdrücke

Reflection

Reguläre Ausdrücke

Consumer

Streams

Szenarien zu Regulären Ausdrücken

Zerlegen einer Zeichenfolge

- Zeichenfolgen werden an bestimmten Trennstellen in mehrere Teilfolgen zerlegt. Die Trennstellen werden durch ein Muster vorgegeben.
- Die Methode split(String regex) der Klasse String gibt ein String[ ]-Objekt zurück, die Teilfolgen der Zeichenfolge, getrennt nach dem vorgegebenen Regulären Ausdruck regex.

```
public final class String … {
    …
    public String[ ] split(String regex) {
        return split(regex, 0);
    }

    public String[ ] split(Strin regex, int limit){
        …
        return Pattern.compile(regex)
            .split(this, limit);
    }
}
```

Reguläre Ausdrücke

Reflection

Reguläre Ausdrücke

Consumer

Streams

Aufgabe, Teil 3 zu Regulären Ausdrücken

- Gegeben sei die folgende Klasse Anwendung. Ergänzen Sie diese, so dass Sie die folgende Ausgabe in der Konsole erhalten.

Wie erreichen Sie eine Zerlegung mittels Punkt, also . als Trennzeichen?

```
Das Ergebnis nach Zerlegen der Zeichenkette mittels
Trennzeichen , ist
Liebe Studierende |  wie geht es Ihnen? |
```

```
public class Anwendung {
    private String zeichenkette;
    // Konstruktor, welcher zeichenkette belegt

    private void zerlegeZeichenkette(String regex){
        // hier ergaenzen
        …
    }

    public static void main(String[ ] args){
        Anwendung anw = new Anwendung(
            "Liebe Studierende, wie geht es Ihnen?")
        anw.zerlegeZeichenkette(",");
    }
}
```

Reguläre Ausdrücke

Wieso Reguläre Ausdrücke?

- vgl.: http://openbook.rheinwerk-verlag.de

  Reguläre Ausdrücke werden bei der Zeichenketten-verarbeitung beim Suchen und Ersetzen eingesetzt.

Reflection

Reguläre Ausdrücke

Consumer

Streams

Consumer

Reflection

Reguläre Ausdrücke

Consumer

Streams

Wiederholung: Supplier<T>

- Funktionales Interface, welches etwas anbietet:

```
@FunctionalInterface
public interface Supplier<T>{
    T get();
}
```

Consumer<T>

- Funktionales Interface, welches etwas konsumiert:

```
@FunctionalInterface
public interface Consumer<T>{

    void accept(T t);

    default Consumer<T> andThen(
        Consumer<?  super T> after){
        Objects.requireNonNull(after);
        return (T t) -> {
            accept(t); after.accept(t);
        };
    }
}
```

Consumer

Einschub: default – Methoden in Interfaces

- Gegeben seien das Interface Begruessen und die Klasse Anwendung.

```
public interface Begruessen{
    String getGruss(String tageszeit);
}
```

```
public class Anwendung{
    public static void main(String[ ] args){
        Begruessen gruss = (e -> "Guten " + e + "!");
        System.out.println(gruss.getGruss("Morgen"));
    }
}
```

- **Wunsch**
  Jetzt soll das Interface noch eine Methode erhalten für den Fall, dass die Tageszeit unbekannt ist, mit dem Gruß Hallo!.
  Bedingung: Das bisherige Programm, welches Begruessen benutzt, soll ohne Veränderung fehlerfrei weiterlaufen!

Consumer

Einschub: default – Methoden in Interfaces

- **Lösung ohne default-Methode**

```
public interface BegruessenPlus
      extends Begruessen{
      String getGruss();
}
```

```
public class Anwendung{
     public static void main(String[ ] args){
          Begruessen gruss = (e -> "Guten " + e + "!");
          System.out.println(gruss.getGruss("Morgen"));
          BegruessenPlus grussPlus = new BegruessenPlus(){
               public String getGruss(String tageszeit){
                    return "Guten " + tageszeit + "!");
               }
               public String getGruss(){
                    return "Hallo!";
               }
          };
          System.out.println(grussPlus.getGruss());
     }
}
```

Reflection

Reguläre Ausdrücke

Consumer

Streams

Consumer

Einschub: default – Methoden in Interfaces

- **Lösung mit default-Methode**

Reflection

Reguläre Ausdrücke

Consumer

Streams

```
public interface Begruessen{
    String getGruss(String tageszeit);
    default String getGruss(){
        return "Hallo!";
    }
}
```

Begruessen ist immer noch ein funktionales Interface!

```
public class Anwendung{
    public static void main(String[ ] args){
        Begruessen gruss = (e -> "Guten " + e + "!");
        System.out.println(gruss.getGruss("Morgen"));
        System.out.println(gruss.getGruss());
    }
}
```

Consumer

Reflection

Reguläre Ausdrücke

Consumer

Streams

Consumer<T>

- default-Methode andThen

```
@FunctionalInterface
public interface Consumer<T>{

    void accept(T t);

    default Consumer<T> andThen(
        Consumer<?  super T> after){
        Objects.requireNonNull(after);
        return (T t) -> {
            accept(t); after.accept(t);
        };
    }
}
```

Beispiel

```
public class Anwendung{

    public static void main(String[ ] args){
        Consumer<String> gruessen
            = (e -> System.out.println(
            "Guten " + e + "!"));
        gruessen.accept("Morgen");
    }
}
```

Frage

- Welche Ausgabe erhalten Sie durch den folgenden Aufruf?
  gruessen.andThen(gruessen).accept("Tag");

Consumer

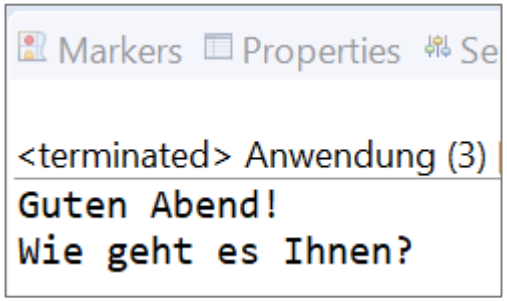
Aufgabe, Teil 1 zu Consumer<T>

- Erstellen Sie eine Klasse Anwendung, so dass Sie die vorgegebene Ausgabe erhalten. Verwenden Sie Consumer<String>.

```
@FunctionalInterface
public interface Consumer<T>{

    void accept(T t);

    default Consumer<T> andThen(
        Consumer<?  super T> after){
        Objects.requireNonNull(after);
        return (T t) -> {
            accept(t); after.accept(t);
        };
    }
}
```



Reflection
Reguläre Ausdrücke
Consumer
Streams

Consumer

Aufgabe, Teil 2 zu Consumer<T>

- Gegeben sie die folgende Klasse Person. Erstellen Sie eine Klasse Anwendung, so dass Sie die vorgegebene Ausgabe erhalten. Verwenden Sie Consumer<Person>.

Reflection

Reguläre Ausdrücke

Consumer

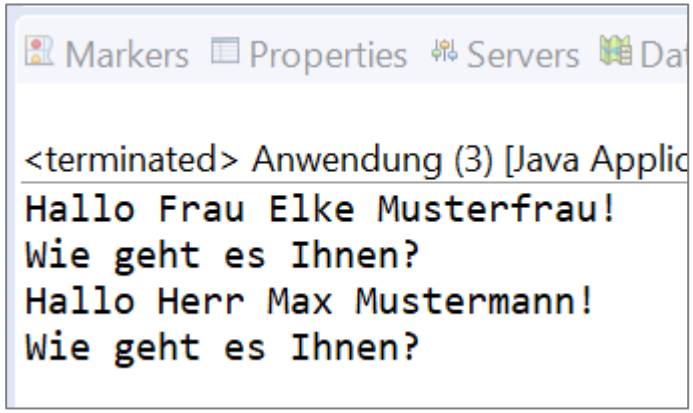
Streams


```
public class Person {

    private String vorname;
    private String nachname;
    private String anrede;

    // Konstruktor, welcher die
    // Attribute mit Werten belegt,
    // und get- uns set-Methoden
    …

}
```



Consumer

- **Wunsch**
  - Komfortable Verarbeitung von Mengen von Objekten.
- **Beispiel: for und for each**
  - vor Java 5:

```
List nachnamen
    = Arrays.asList("Musterfrau", "Mustermann", "Musterperson");
for(int i = 0; i < nachnamen.size(); i++){
    System.out.println((String)nachnamen.get(i));
}
```

  - seit Java 5:

```
List<String> nachnamen
    = Arrays.asList("Musterfrau", "Mustermann", "Musterperson");
for(String nachname : nachnamen){
    System.out.println(nachname);
}
```

Reflection
Reguläre Ausdrücke
Consumer
Streams

Consumer

- **Beispiel für eine komfortable Verarbeitung von Mengen**
  - seit Java 5 / Java 8:

```
List<String> nachnamen
    = Arrays.asList("Musterfrau", "Mustermann", "Musterperson");
nachnamen.forEach(nachname -> System.out.println(nachname));
```


Reflection

Reguläre Ausdrücke

Consumer

Streams

- Die default-Methode forEach des Interfaces Iterable<T> prüft den vorgegebenen Consumer-Parameter auf nicht-null und führt ggf. die Implementierung von accept des Consumers mit jedem Element aus Iterable aus.

Implementierung des Interfaces Consumer<T>

```
default void forEach(Consumer<? super T> action) {
    Objects.requireNonNull(action);
    for (T t : this) {
        action.accept(t);
    }
}
```

Consumer

Wieso Consumer?

- Consumer-Typen sind gedacht als Endglied einer Kette. Beispielsweise werden bereits verarbeitete Daten in eine Datei geschrieben.
- vgl.: *https://docs.oracle.com/javase/8/docs/api/java/util/function/Consumer.html*
  Unlike most other functional interfaces, Consumer is expected to operate via side-effects.
- Beispiel:
  forEach aus Iterable benötigt als Parameter ein Consumer-Objekt.

Reflection
Reguläre Ausdrücke
Consumer
Streams

Streams

Reflection

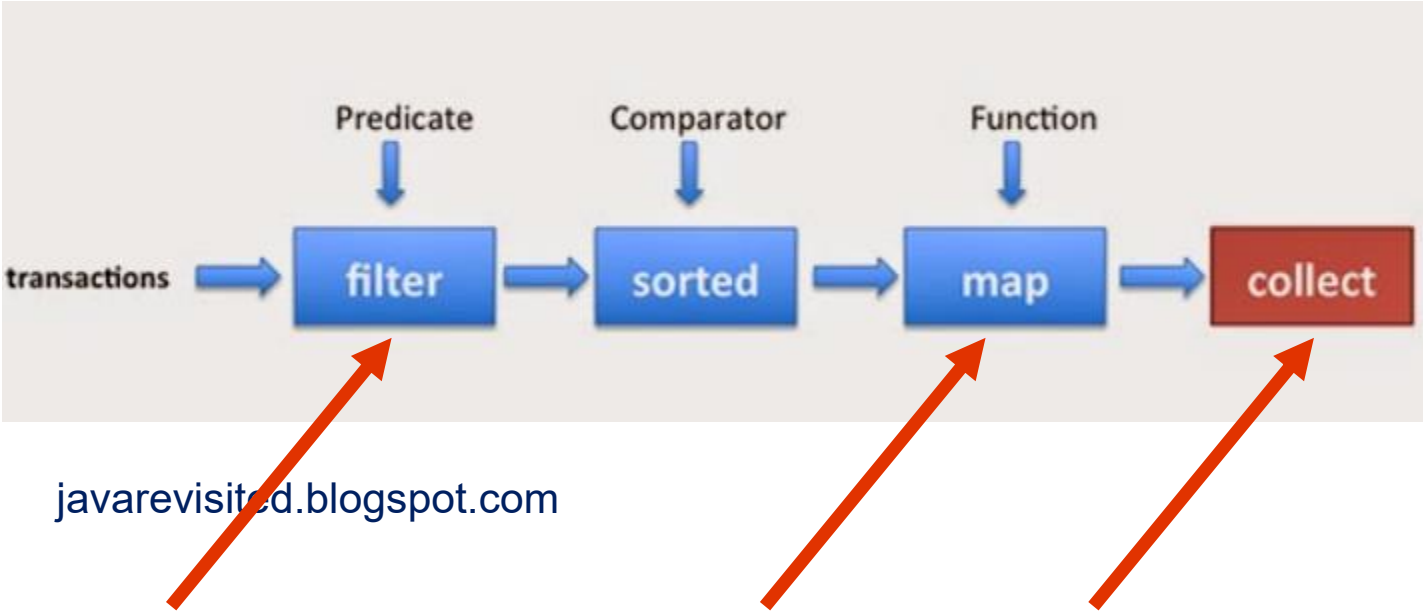
Reguläre Ausdrücke

Consumer

Streams

Was sind Streams?

- Stream<E> ist ein Interface, mit dessen Hilfe Mengen von Objekten komfortabel verarbeitet werden können.



javarevisited.blogspot.com

Streams

Reflection

Reguläre Ausdrücke

Consumer

Streams

Streams: Filtern und Zählen

- Stream<E> ist ein Interface, mit dessen Hilfe Mengen von Objekten komfortabel verarbeitet werden können.
- Die default Methode stream() des Interfaces Collection<E> gibt ein Stream<E> - Objekt zurück.
- Die Methode filter(Predicate<? Super T> predicate) des Interfaces Stream<T> gibt ein Stream<T> - Objekt zurück, welches diejenigen Elemente des Streams enthält, die die Bedingung des Parameters predicate erfüllen.
- Predicate<T> ist ein funktionales Interface mit der Methode boolean test(T t); und weiteren default-Methoden, welche logische Operationen implementieren.
- Die Methode count() des Interfaces Stream<T> gibt einen long - Wert zurück.

Streams

Reflection

Reguläre Ausdrücke

Consumer

Streams

Streams: Sammeln

- Die Methode
  <R, A> R collect(Collector<? super T, A, R> collector) des Interfaces Stream<T> sammelt die Elemente des vorliegenden Stream<T> - Objekts und gibt diese in Form R des vorgegebenen Parameters collector zurück:
  Collectors.toList() -> als List<T>,
  Collectors.joining(CharSequence delimiter) -> als String, in welchem die Elemente des vorliegenden Stream<T> - Objekts hintereinandergehängt werden, durch den vorgegebenen delimiter voneinander getrennt.
- Collectors ist eine finale Klasse.

Streams

Aufgabe, Teil 1 zu Streams

vgl.: https://www.java67.com/2014/04/java-8-stream-examples-and-tutorial.html

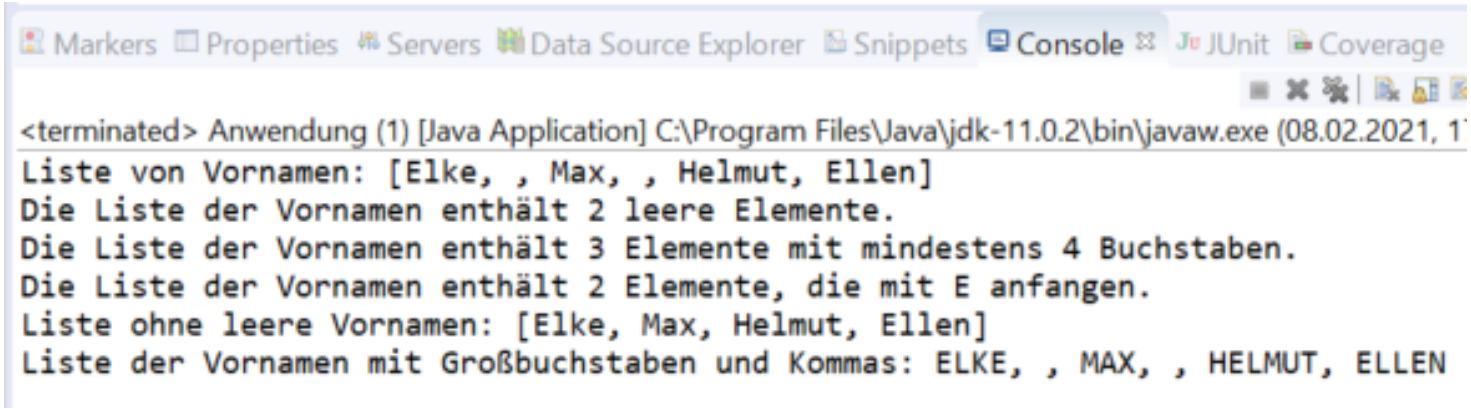
Ergänzen Sie den Quellcode aus dem ersten Beispiel, so dass Sie die folgende Ausgabe erhalten.

Reflection

Reguläre Ausdrücke

Consumer

Streams



Streams

Streams: Bearbeiten

- Die Methode
  <R> Stream<R> map(Function<? super T, ? extends R> mapper)
  des Interfaces Stream<T> gibt ein Stream<R> - Objekt zurück, welches Elemente enthält, die man erhält, indem man die durch den Parameter mapper gegebene Funktion auf die Elemente anwendet.
- Function<T, R> ist ein funktionales Interface mit der Methode R apply(T t) und weiteren default-Methoden.
- Die Methode distinct() des Interfaces Stream<T> gibt ein Stream<T> - Objekt zurück, dessen Elemente sich voneinander unterscheiden (sollen).

Reflection
Reguläre Ausdrücke
Consumer
Streams

Streams

Reflection

Reguläre Ausdrücke

Consumer

Streams

Fortsetzung zu Streams: Bearbeiten

- Die Klasse IntSummaryStatistics implementiert das funktionale Interface IntConsumer, welches die Methode void accept(int value) und eine weitere default – Methode enthält.
- Methoden der Klasse IntSummaryStatistics:
  public final int getMax()
  public final int getMin()
  public final long getSum()
  public final int getAverage() und weitere …
- Aus einem Objekt listeZahlen vom Typ List<Integer> erhält man ein Objekt neueListeZahlen vom Typ IntSummaryStatistics mittels
  neueListeZahlen
  = listeZahlen.stream().mapToInt((x) -> x).summaryStatistics();

Streams

Reflection

Reguläre Ausdrücke

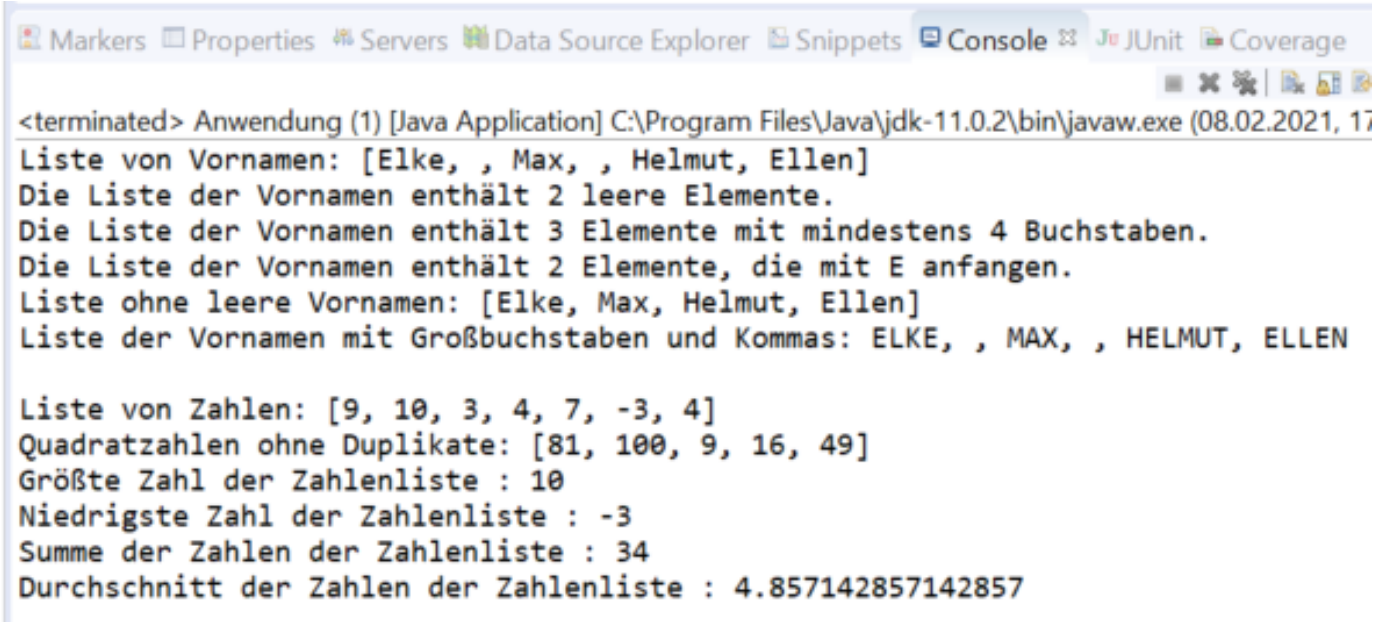
Consumer

Streams

Aufgabe, Teil 2 zu Streams

vgl.: https://www.java67.com/2014/04/java-8-stream-examples-and-tutorial.html

Ergänzen Sie den Quellcode aus dem vorliegenden Beispiel, so dass Sie die folgende Ausgabe erhalten.



Streams

Wieso Streams?

vgl.: https://www.java67.com/2014/04/java-8-stream-examples-and-tutorial.html

- Bearbeitung von Mengen von Objekten innerhalb von Containern unter Verwendung von Techniken aus Java 8
- Lazy Evaluation: Ausdrücke werden nur so weit ausgewertet, wie es für das Ergebnis notwendig ist.

Reflection

Reguläre Ausdrücke

Consumer

Streams

sonstiges

- Reflection ✓
- Reguläre Ausdrücke ✓
- Consumer ✓
- Streams ✓

Hochschule Bochum
TECHNIK WIRTSCHAFT GESUNDHEIT

Objektorientierte Programmiertechniken

Vielen Dank für Ihr Interesse!